



UNIVERSIDAD DIEGO PORTALES

Laboratorio 4: Árboles de búsqueda

Inventario del Club de Robótica

ESTRUCTURAS DE DATOS Y ALGORITMOS
2026-1

Profesores:
Matías Aguilera
Marcos Fantóval
Cristián Llull
Karol Suchan

Trabajo en equipos de 3 estudiantes

1. Introducción

El Club de Robótica de la universidad administra una gran cantidad de componentes electrónicos y mecánicos: sensores, motores, placas Arduino, Raspberry Pi, cables, baterías, protoboards, módulos de comunicación, herramientas y kits de desarrollo.

A medida que el club crece, aumenta el número de operaciones que se realizan sobre el inventario. Cada semana se registran compras de nuevos materiales, préstamos a estudiantes, devoluciones de componentes y bajas de elementos dañados, perdidos u obsoletos.

Para administrar este inventario de forma eficiente, se necesita una estructura de datos que permita almacenar, consultar, actualizar y eliminar elementos con rapidez.

En este laboratorio, usted y su equipo deberán implementar un sistema simplificado de inventario para el Club de Robótica, utilizando tablas de símbolos basadas en árboles de búsqueda.

El objetivo principal es comparar experimental y teóricamente el desempeño de dos estructuras del texto guía de Sedgewick y Wayne:

- `BST<Key, Value>`
- `RedBlackBST<Key, Value>`

Ambas permiten asociar una clave a un valor, pero se comportan de manera distinta. `BST` corresponde a un árbol binario de búsqueda no necesariamente balanceado, mientras que `RedBlackBST` mantiene el balance mediante reglas de árboles rojo-negro.

1.1. Motivación

Imagine que el Club de Robótica necesita responder constantemente a operaciones como:

- Comprar nuevos componentes.
- Consultar si hay stock disponible.
- Prestar componentes a estudiantes.
- Registrar devoluciones.
- Dar de baja los componentes dañados o perdidos.

En términos de una tabla de símbolos, estas acciones pueden modelarse mediante operaciones como:

```
1 put(id, item)
2 get(id)
3 delete(id)
```

La pregunta central del laboratorio es:

¿Qué estructura de datos es más conveniente para administrar un inventario dinámico sometido a una larga secuencia de compras, consultas, préstamos, devoluciones y bajas?

Para responder esta pregunta, deberán realizar experimentos controlados, medir los tiempos de ejecución, analizar la altura de los árboles y comparar los resultados con la teoría Big-O.

1.2. Objetivos

Al finalizar el laboratorio, el estudiante será capaz de:

1. Usar las clases del texto guía de Princeton para resolver un problema aplicado.
2. Modelar un sistema de inventario mediante pares clave-valor.
3. Comparar y analizar empíricamente la diferencia en el desempeño de BST y RedBlackBST.
4. Documentar los resultados mediante archivos CSV, gráficos, tablas y un informe técnico.

2. Base de código obligatoria

Este laboratorio debe construirse usando la biblioteca `algs4` del texto guía.

Debe importar y utilizar obligatoriamente las siguientes clases de Princeton:

- `BST`
- `RedBlackBST`
- `StdRandom`
- `StopwatchCPU`
- `Out`

No debe reimplementar desde cero ni `BST` ni `RedBlackBST`.

Referencias del texto guía:

- <https://algs4.cs.princeton.edu/code/javadoc/edu/princeton/cs/algs4/BST.html>
- <https://algs4.cs.princeton.edu/code/javadoc/edu/princeton/cs/algs4/RedBlackBST.html>
- <https://algs4.cs.princeton.edu/code/javadoc/edu/princeton/cs/algs4/StdRandom.html>
- <https://algs4.cs.princeton.edu/code/javadoc/edu/princeton/cs/algs4/StopwatchCPU.html>
- <https://algs4.cs.princeton.edu/code/javadoc/edu/princeton/cs/algs4/Out.html>

3. Especificaciones del sistema

Debe implementar un sistema de inventario para el Club de Robótica. El sistema trabajará con:

1. Una clase `InventoryItem`.
2. Una clase `InventoryOperation`.
3. Un generador de datos.
4. Dos adaptadores para las estructuras del texto guía.
5. Una clase principal de experimentación.

3.1. Clase InventoryItem

La clase `InventoryItem` representa un elemento del inventario.

Debe tener los siguientes atributos:

Atributo	Tipo	Descripción
<code>id</code>	<code>int</code>	Identificador único del elemento.
<code>name</code>	<code>String</code>	Nombre del componente.
<code>category</code>	<code>String</code>	Categoría del componente.
<code>location</code>	<code>String</code>	Ubicación física dentro del club.
<code>stockTotal</code>	<code>int</code>	Cantidad total registrada.
<code>stockAvailable</code>	<code>int</code>	Cantidad disponible para préstamo.
<code>stockOnLoan</code>	<code>int</code>	Cantidad actualmente prestada.

Constructor obligatorio:

```
1 public InventoryItem(  
2     int id,  
3     String name,  
4     String category,  
5     String location,  
6     int stockTotal,  
7     int stockAvailable,  
8     int stockOnLoan  
9 )
```

Debe implementar getters para todos los atributos.

También debe implementar métodos que permitan modificar el inventario, por ejemplo (no excluyente):

```
1 public void addStock(int itemId, int quantity)  
2 public boolean lend(int itemId, int quantity)  
3 public boolean receive(int itemId, int quantity)
```

Reglas mínimas:

- No puede existir stock negativo.
- No se puede prestar más de lo disponible.
- No se puede devolver más de lo prestado actualmente.
- Si se compra más stock de un componente existente, debe aumentar `stockTotal` y `stockAvailable`.
- Si se presta un componente, debe disminuir `stockAvailable` y aumentar `stockOnLoan`.
- Si se devuelve un componente, debe aumentar `stockAvailable` y disminuir `stockOnLoan`.

3.2. Tipos de operaciones

Debe crear un enum llamado `OperationType` con los siguientes valores:

```

1 public enum OperationType {
2     PURCHASE,
3     QUERY,
4     LEND,
5     RECEIVE,
6     DISPOSE
7 }

```

Cada operación representa una acción real del Club de Robótica:

Operación	Significado	Operación base en tabla de símbolos
PURCHASE	Compra o ingreso de componentes.	put
QUERY	Consulta de disponibilidad.	get
LEND	Préstamo de componentes.	get + actualización
RECEIVE	Devolución de componentes.	get + actualización
DISPOSE	Baja por daño, pérdida u obsolescencia.	delete

3.3. Clase InventoryOperation

La clase `InventoryOperation` representa una operación que será ejecutada sobre el inventario.

Debe tener los siguientes atributos:

Atributo	Tipo	Descripción
<code>type</code>	<code>OperationType</code>	Tipo de operación.
<code>key</code>	<code>int</code>	ID del componente.
<code>quantity</code>	<code>int</code>	Cantidad involucrada en la operación.
<code>item</code>	<code>InventoryItem</code>	Elemento asociado. Solo se usa directamente en compras nuevas.

Constructor sugerido:

```

1 public InventoryOperation(
2     OperationType type,
3     int key,
4     int quantity,
5     InventoryItem item
6 )

```

En operaciones `QUERY`, `LEND`, `RECEIVE` y `DISPOSE`, el atributo `item` debe ser `null`.

3.4. Interfaz InventoryIndex

Para comparar ambas estructuras de forma uniforme, debe crear una interfaz común:

```

1 public interface InventoryIndex {
2     void put(Integer key, InventoryItem value);
3     InventoryItem get(Integer key);
4     void delete(Integer key);
5     boolean contains(Integer key);

```

```
6     Iterable<Integer> keys();
7     int size();
8     int height();
9 }
```

Esta interfaz permitirá ejecutar la misma secuencia de operaciones sobre BST y RedBlackBST.

3.5. Clase BSTInventoryIndex

Debe implementar la interfaz `InventoryIndex` usando internamente:

```
1 private BST<Integer, InventoryItem> st;
```

La clase debe delegar sus operaciones en la estructura de Princeton.

Ejemplo:

```
1 public void put(Integer key, InventoryItem value) {
2     st.put(key, value);
3 }
```

3.6. Clase RedBlackBSTInventoryIndex

Debe implementar la interfaz `InventoryIndex` usando internamente:

```
1 private RedBlackBST<Integer, InventoryItem> st;
```

La clase debe delegar sus operaciones en la estructura de Princeton.

3.7. Clase DataGenerator

Debe implementar una clase `DataGenerator` que genere operaciones aleatorias y reproducibles sobre el inventario.

3.7.1. Generación de componentes

Debe implementar el siguiente método:

```
1 public static InventoryItem generateItem(int id)
```

Este método debe generar un componente con los siguientes criterios:

- `id`: valor recibido como parámetro.
- `name`: `Componente + id`.
- `category`: seleccionada aleatoriamente desde un arreglo fijo.
- `location`: seleccionada aleatoriamente desde un arreglo fijo.
- `stockTotal`: entero aleatorio entre 1 y 20.

- `stockAvailable`: inicialmente igual a `stockTotal`.
- `stockOnLoan`: inicialmente igual a 0.

Categorías sugeridas:

```

1 String[] categories = {
2     "Sensor",
3     "Motor",
4     "Microcontrolador",
5     "Cable",
6     "Bateria",
7     "Herramienta",
8     "Modulo",
9     "Kit"
10 };

```

Ubicaciones sugeridas:

```

1 String[] locations = {
2     "Estante_A",
3     "Estante_B",
4     "Caja_1",
5     "Caja_2",
6     "Laboratorio",
7     "Bodega"
8 };

```

3.7.2. Generación de operaciones

Debe implementar el siguiente método:

```

1 public static ArrayList<InventoryOperation> generateOperations(
2     int m,
3     int keyUniverse,
4     long seed
5 )

```

Donde:

Parámetro	Descripción
<code>m</code>	Cantidad total de operaciones a generar.
<code>keyUniverse</code>	Rango de claves posibles. Las claves estarán entre 1 y <code>keyUniverse</code> .
<code>seed</code>	Semilla para reproducibilidad.

Debe usar:

```

1 StdRandom.setSeed(seed);

```

para asegurar que los experimentos sean reproducibles.

La distribución obligatoria de operaciones será:

Operación	Probabilidad
PURCHASE	0.35
QUERY	0.30
LEND	0.15
RECEIVE	0.10
DISPOSE	0.10

Cada operación debe seleccionar una clave aleatoria en el rango:

```
1 [1, keyUniverse]
```

Para las operaciones PURCHASE, LEND y RECEIVE, la cantidad asociada debe ser un entero aleatorio entre 1 y 5. Para las operaciones QUERY y DISPOSE debe ser 0.

En el caso de PURCHASE, si el componente no está en el inventario (en base a la **key** escogida al azar), debe crearse un nuevo `InventoryItem` y pasarse en el atributo `item` de la `InventoryOperation` creada. Si está, el atributo `item` de la `InventoryOperation` debe ser `null`. Para poder ejecutarlo correctamente, el método `generateOperations` debe manejar internamente el conjunto de los id de los componentes presentes en el inventario en cada momento de su ejecución.

4. Etapas del laboratorio

4.1. Etapa 1: Implementación del sistema de inventario

Debe implementar:

1. Clase `InventoryItem`.
2. Clase `InventoryOperation`.
3. Enum `OperationType`.
4. Interfaz `InventoryIndex`.
5. Clase `BSTInventoryIndex`.
6. Clase `RedBlackBSTInventoryIndex`.
7. Clase `DataGenerator`.
8. Clase `Experiment`.

La clase `Experiment` será responsable de:

- Generar las secuencias de operaciones.
- Ejecutar la secuencia sobre `BSTInventoryIndex`.
- Ejecutar la secuencia sobre `RedBlackBSTInventoryIndex`.
- Medir tiempos.
- Guardar los resultados en archivos CSV.
- Verificar que ambas estructuras terminen en estados equivalentes.

4.2. Etapa 2: Experimento principal

El objetivo es comparar el tiempo de ejecución con BST y RedBlackBST sobre una larga secuencia aleatoria de operaciones de inventario.

Para cada tamaño $m = 2^t$, con $t \in 12, 13, 14, 15, 16, 17, 18, 19$, es decir:

t	m
12	4.096
13	8.192
14	16.384
15	32.768
16	65.536
17	131.072
18	262.144
19	524.288

Repita el experimento sobre 30 instancias diferentes para cada tamaño.

Para cada instancia i , use:

```
1 seed = m + i
```

y:

```
1 keyUniverse = 4 * m
```

4.2.1. Procedimiento experimental

Para cada tamaño m y para cada instancia i :

1. Genere una lista de m operaciones usando `generateOperations`.
2. Cree una estructura `BSTInventoryIndex` vacía.
3. Cree una estructura `RedBlackBSTInventoryIndex` vacía.
4. Ejecute la secuencia sobre `BSTInventoryIndex`.
5. Mida el tiempo total usando `StopwatchCPU`.
6. Repita el mismo proceso sobre `RedBlackBSTInventoryIndex`.
7. Registre los resultados en un archivo CSV.

Durante la ejecución de las operaciones, debe aplicar las siguientes reglas.

Compra: PURCHASE

- Si la clave no existe:

```
1 put(key, item);
```

- Si la clave ya existe:

```
1 InventoryItem item = get(key);
2 item.addStock(quantity);
3 put(key, item);
```

Consulta: QUERY

- Debe ejecutar:

```
1 get(key);
```

Debe registrar si la consulta fue exitosa o fallida.

Préstamo: LEND

- Debe ejecutar:

```
1 InventoryItem item = get(key);
```

Si el elemento existe y tiene stock disponible suficiente, debe registrar el préstamo.

Si no existe o no hay stock suficiente, la operación no se ejecuta y se considera fallida.

Devolución: RECEIVE

- Debe ejecutar:

```
1 InventoryItem item = get(key);
```

Si el elemento existe y tiene elementos prestados suficientes, debe registrar la devolución.

Si no existe o no hay elementos prestados suficientes, la operación no se ejecuta y se considera fallida.

Baja: DISPOSE

- Debe ejecutar:

```
1 delete(key);
```

La baja elimina por completo el componente del inventario. Representa el caso en el que el componente queda obsoleto, se pierde o se decide retirarlo del sistema.

4.2.2. Datos que debe registrar

Por cada ejecución debe registrar:

Campo	Descripción
instancia	Número de repetición.
estructura	BST o RedBlackBST.
m	Cantidad total de operaciones.

<code>purchase_total</code>	Cantidad de compras.
<code>query_total</code>	Cantidad de consultas.
<code>lend_total</code>	Cantidad de préstamos.
<code>receive_total</code>	Cantidad de devoluciones.
<code>dispose_total</code>	Cantidad de bajas.
<code>query_successful</code>	Consultas exitosas.
<code>query_failed</code>	Consultas fallidas.
<code>lend_successful</code>	Préstamos exitosos.
<code>lend_failed</code>	Préstamos fallidos.
<code>receive_successful</code>	Devoluciones exitosas.
<code>receive_failed</code>	Devoluciones fallidas.
<code>final_size</code>	Cantidad final de claves en la estructura.
<code>final_height</code>	Altura final del árbol.
<code>elapsed_seconds</code>	Tiempo total de ejecución.

4.2.3. Ejemplo de medición

La medición debe realizarse solo sobre la ejecución de las operaciones.

En particular, no debe incluirse en el tiempo medido:

- la generación de operaciones;
- la creación de la lista;
- la escritura del CSV;
- la impresión en consola.

Ejemplo:

```

1 StopwatchCPU timer = new StopwatchCPU();
2
3 for (InventoryOperation op : operations) {
4     if (op.getType() == OperationType.PURCHASE) {
5         executePurchase(index, op);
6     } else if (op.getType() == OperationType.QUERY) {
7         executeQuery(index, op);
8     } else if (op.getType() == OperationType.LEND) {
9         executeLend(index, op);
10    } else if (op.getType() == OperationType.RECEIVE) {
11        executeReceive(index, op);
12    } else if (op.getType() == OperationType.DISPOSE) {
13        executeDispose(index, op);
14    }
15 }
16
17 double elapsed = timer.elapsedTime();

```

4.2.4. Formato del archivo CSV

Debe generar un archivo CSV por cada tamaño.

Ejemplo:

```
1 inventory_experiment_4096.csv
2 inventory_experiment_8192.csv
3 inventory_experiment_16384.csv
4 ...
```

Cada archivo debe tener el siguiente encabezado:

```
1 instancia,estructura,m,purchase_total,query_total,lend_total,receive_total,
  dispose_total,query_successful,query_failed,lend_successful,lend_failed,
  receive_successful,receive_failed,final_size,final_height,elapsed_seconds
```

4.3. Etapa 3: Validación de correctitud

Además de medir tiempos, debe validar que ambas estructuras procesan correctamente la secuencia de operaciones.

Para cada instancia, después de terminar la ejecución de las operaciones, debe verificar:

1. Que el tamaño final de `BSTInventoryIndex` y `RedBlackBSTInventoryIndex` sea el mismo.
2. Que para una muestra de 100 claves aleatorias, ambas estructuras retornen el mismo resultado con `get`.
3. Que el número de operaciones ejecutadas sea exactamente `m`.
4. Que no existan cantidades de stock negativas.
5. Que `stockAvailable` + `stockOnLoan` sea igual a `stockTotal` para cada elemento activo.
6. Que no existan excepciones por claves nulas o valores nulos.

Si alguna validación falla, el experimento debe reportar el error y no debe considerarse válido.

4.4. Etapa 4: Análisis empírico

Debe construir un archivo:

```
1 analisis_inventario_arboles.xlsx
```

El archivo debe contener las pestañas descritas a continuación.

4.4.1. Pestañas por tamaño

Una pestaña por cada tamaño:

```
1 Inv_4096
2 Inv_8192
3 Inv_16384
4 Inv_32768
5 Inv_65536
6 Inv_131072
7 Inv_262144
8 Inv_524288
```

En cada pestaña debe incluir:

1. Microdatos de las 30 instancias para BST.
2. Microdatos de las 30 instancias para RedBlackBST.
3. Estadísticas para cada estructura:
 - mínimo;
 - máximo;
 - promedio;
 - desviación estándar;
 - coeficiente de variación;
 - altura promedio;
 - altura máxima.
4. Un histograma de tiempos para BST.
5. Un histograma de tiempos para RedBlackBST.

4.4.2. Pestaña Resumen

La pestaña **Resumen** debe incluir:

1. Tabla de tiempo promedio por tamaño y por estructura.
2. Tabla de altura promedio por tamaño y por estructura.
3. Tabla de cantidad promedio de operaciones exitosas y fallidas.
4. Gráfico log-log de tiempo promedio:
 - eje x: tamaño m ;
 - eje y: tiempo promedio;
 - una curva para BST;
 - una curva para RedBlackBST.
5. Gráfico de altura promedio:
 - eje x: tamaño m ;
 - eje y: altura promedio;
 - una curva para BST;
 - una curva para RedBlackBST.
6. Cálculo de ratios entre tamaños consecutivos:

```
1 ratio = tiempo_promedio(2m) / tiempo_promedio(m)
```

También debe calcular:

```
1 log2(ratio)
```

Finalmente, debe incluir una interpretación breve:

- ¿A qué valor parece acercarse $\log_2(\text{ratio})$?
- ¿El crecimiento observado se parece a $O(\log n)$, $O(n)$, $O(n \log n)$ u otro?
- ¿La altura del árbol ayuda a explicar los tiempos obtenidos?
- ¿Qué estructura parece más estable para administrar el inventario?

4.5. Etapa 5: Análisis teórico

El informe debe incluir un análisis teórico para ambas estructuras.

Para BST, debe explicar:

1. Qué es una tabla de símbolos.
2. Por qué la forma del árbol depende del orden de inserción.
3. Qué se puede decir sobre las alturas de los árboles obtenidos.
4. Cuáles son las complejidades asintóticas de los métodos utilizados.

Para RedBlackBST, debe explicar:

1. Qué problema intenta resolver respecto de BST.
2. Qué se puede decir sobre las alturas de los árboles obtenidos.
3. Cuáles son las complejidades asintóticas de los métodos utilizados.

4.6. Etapa 6: Comparación teoría vs. experimentación

Debe comparar los resultados empíricos con lo esperado teóricamente.

Para cada estructura responda:

1. ¿Cómo crece el tiempo promedio cuando se duplica m ?
2. ¿El gráfico log-log coincide con la complejidad esperada?
3. ¿Cómo cambia la altura promedio del árbol?
4. ¿Existe una relación visible entre la altura y el tiempo?
5. ¿Cuál estructura recomendaría para el inventario del Club de Robótica y por qué?

5. Informe

Además del código y del archivo de Excel, debe entregar un informe en formato PDF.

El informe debe tener la siguiente estructura obligatoria.

5.1. Introducción

Debe incluir:

- Descripción general del problema.
- Motivación del Club de Robótica.
- Objetivos del laboratorio.
- Estructuras comparadas.

5.2. Implementación

Debe incluir:

- Descripción de la clase `InventoryItem`.
- Descripción de la clase `InventoryOperation`.
- Descripción de `InventoryIndex`.
- Descripción de `BSTInventoryIndex`.
- Descripción de `RedBlackInventoryIndex`.
- Descripción del generador de operaciones.
- Explicación de cómo se aseguró de que ambas estructuras ejecutaran exactamente la misma secuencia.

5.3. Diseño experimental

Debe incluir:

- Tamaños usados.
- Cantidad de instancias por tamaño.
- Distribución de operaciones.
- Uso de semillas.
- Qué se midió y qué no se midió.
- Formato de los archivos CSV.
- Validaciones de correctitud.

5.4. Resultados empíricos

Debe incluir:

- Tablas resumen.
- Gráficos log-log.
- Histogramas.
- Comparación de alturas.
- Comentarios sobre variabilidad.
- Discusión sobre la estabilidad del tiempo de ejecución.

5.5. Análisis teórico

Debe incluir:

- Big-O de put, get y delete en BST.
- Big-O de put, get y delete en RedBlackBST.
- Diferencia entre el caso promedio y el peor caso.
- Relación entre la altura del árbol y el costo de operación.

5.6. Comparación teoría vs. práctica

Debe incluir:

- Comparación entre los tiempos observados y la complejidad teórica.
- Interpretación de ratios.
- Interpretación de $\log_2(\text{ratio})$.
- Discusión sobre si los datos empíricos respaldan la teoría.

5.7. Conclusiones

Debe responder:

1. ¿Qué estructura tuvo el mejor desempeño promedio?
2. ¿Qué estructura tuvo la menor variabilidad?
3. ¿Cuál mantuvo menor altura?
4. ¿Cuál conviene usar en una aplicación real de inventario?
5. ¿Qué aprendió el equipo sobre los árboles balanceados?
6. ¿Por qué es importante comparar la teoría y la experimentación?

6. Entregables

Cada equipo debe entregar una carpeta comprimida .zip que contenga:

1. Código fuente completo en Java.
2. Archivos CSV generados.
3. Archivo `analisis_inventario_arboles.xlsx`.
4. Informe en PDF.
5. Archivo `README.txt` con instrucciones para compilar y ejecutar.

La carpeta debe tener la siguiente estructura:

```
1 grupoXX_lab4_arboles/  
2 |  
3 |-- src/  
4 |   |-- InventoryItem.java  
5 |   |-- InventoryOperation.java  
6 |   |-- OperationType.java  
7 |   |-- InventoryIndex.java  
8 |   |-- BSTInventoryIndex.java  
9 |   |-- RedBlackInventoryIndex.java  
10 |   |-- DataGenerator.java  
11 |   |-- Experiment.java  
12 |  
13 |-- data/  
14 |   |-- inventory_experiment_4096.csv  
15 |   |-- inventory_experiment_8192.csv  
16 |   |-- ...  
17 |  
18 |-- analisis_inventario_arboles.xlsx  
19 |-- informe.pdf  
20 |-- README.txt
```

7. Reglas importantes

1. No debe reimplementar BST ni RedBlackBST.
2. Debe usar las clases del texto guía.
3. Debe ejecutar la misma secuencia de operaciones sobre ambas estructuras.
4. No debe imprimir en la consola durante la medición.
5. No debe incluir la generación de operaciones dentro del tiempo medido.
6. No debe incluir la escritura del archivo CSV dentro del tiempo medido.
7. Debe usar semillas para que los experimentos sean reproducibles.
8. Debe validar que ambas estructuras entreguen resultados consistentes.
9. Debe evitar cantidades de stock negativas.
10. Debe explicar los resultados, no solo presentar gráficos.
11. Si falta el código, el Excel o el informe, el laboratorio se considera incompleto.

8. Rúbrica de evaluación

Parte	Puntaje
Implementación de clases base, adaptadores y uso correcto de <code>BST</code> y <code>RedBlackBST</code>	10
Generación reproducible de operaciones y validación de correctitud del inventario	10
Ejecución experimental con mediciones correctas y archivos CSV	10
Excel con microdatos, estadísticos, histogramas, gráficos log-log, ratios y alturas	10
Análisis teórico Big-O de <code>put</code> , <code>get</code> y <code>delete</code> para ambas estructuras	10
Informe: estructura, claridad, comparación teoría-experimento, conclusiones y redacción	10
Total	60

9. Recomendaciones finales

1. Comience probando con tamaños pequeños, por ejemplo $m = 100$ o $m = 1.000$.
2. Verifique que ambas estructuras reciban exactamente las mismas operaciones.
3. Revise que los resultados de `get` coincidan entre ambas estructuras.
4. Use una semilla fija durante las pruebas iniciales para depurar.
5. Cuando el programa funcione, ejecute los tamaños oficiales.
6. Evite imprimir dentro del ciclo de operaciones.
7. Analice la altura de los árboles: será una pista clave para explicar los resultados.
8. El foco del laboratorio es conectar la implementación, la medición y el análisis teórico.